DS603: Robust Machine Learning	Autumn 2025
Lecture 11 — 01/10/2025	
Lecturer: Arjun Bhagoji	*Scribe: Sambhav Jain*

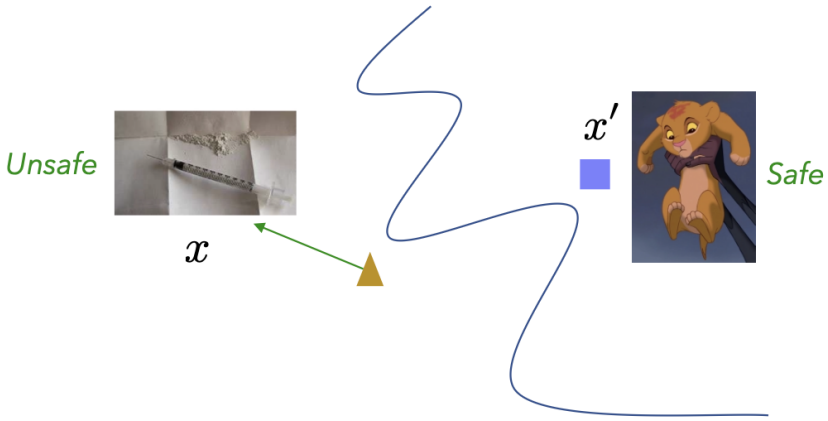
11.1 What is an Adversarial Example?

The key question in adversarial robustness is:

What is a good way for the attacker to perturb samples at test time?

Concept Overview

Adversarial examples are perturbed versions of valid inputs that cause a trained model to misclassify them. For instance, consider an input x that is classified as *unsafe*, and a slightly modified input x' that is visually similar but is classified as *safe*. The perturbation path from x to x' illustrates how small, often imperceptible, changes can fool a model.



We have two main kinds of perturbations:

(i) **ϕ-divergence perturbations:** These adjust sample probabilities.

- Note: Such perturbations are unlikely to lead to misclassification.

(ii) **Optimal transport perturbations:** These lead to strictly stronger attacks than instance-wise perturbations.

Mathematical Formulation

To understand why, we define:

$$L_\varepsilon(\mathscr{H}) := \mathbb{E}_{\tilde{z}=(\mathbf{x},y)\sim P^*}\left[\sup_{c(\mathbf{x},\mathbf{x}_{\text{adv}})\leqslant\varepsilon} \ell\big((\mathbf{x}_{\text{adv}},y),h\big)\right].$$

If we define the Wasserstein ball as:

$$B_\varepsilon(P^*) := \left\{Q \,\middle|\, \inf_{\pi\in\Pi(P^*,Q)} \int_{\mathbf{x}\times\tilde{\mathbf{x}}} c(\mathbf{x},\tilde{\mathbf{x}})\, d\pi(\mathbf{x},\tilde{\mathbf{x}}) \leqslant \varepsilon\right\}.$$

then the following inequality holds:

$$L_\varepsilon(\mathscr{H}) \leqslant \tilde{L}_\varepsilon(\mathscr{H}),$$

where

$$\tilde{L}_\varepsilon(\mathscr{H}) = \sup_{Q\in B_\varepsilon(P^*)} \mathbb{E}_{\tilde{z}\sim Q}[\ell(\tilde{z},h)].$$

Homework: Show that the above inequality holds.

11.2 Neighborhood-Based Attack

Goal: Find a perturbation δ within the neighborhood $N(\cdot)$ such that

$$h(x+\delta) = \text{Target Incorrect Label.}$$

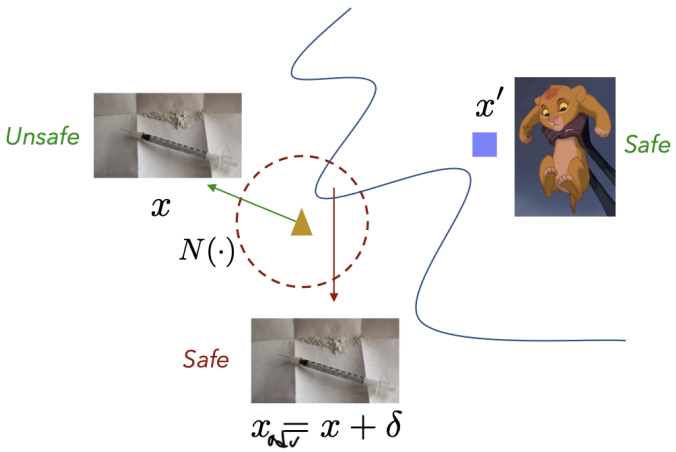


Figure 11.1. Illustration of adversarial perturbation within a neighborhood $N(\cdot)$. The attacker searches for δ within the allowed region so that the model misclassifies $x+\delta$.

Attacker Objective: Maximize the *Adversarial Loss*.

$$L_N(\mathscr{H}) = \mathbb{E}_{z\sim P^*}\left[\sup_{\boldsymbol{x}_{\text{adv}}\in N(\boldsymbol{x})} \ell\big((\boldsymbol{x}_{\text{adv}},y),h\big)\right], \quad \text{where } z=(\mathbf{x},y).$$

If the neighborhood $N(\mathbf{x})$ is defined as a norm ball, i.e.

$$N(\boldsymbol{x}) = \{\boldsymbol{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta} \mid \|\boldsymbol{\delta}\| \leqslant \varepsilon\},$$

then the adversarial loss can be written as

$$L_N(\mathscr{H}) = \mathbb{E}_{z\sim P^*}\left[\sup_{\|\boldsymbol{\delta}\|\leqslant\varepsilon} \ell\big((\mathbf{x}+\boldsymbol{\delta}, y), h\big)\right].$$

For **0-1 Loss** function

$$L_N(\mathscr{H}) = \mathbb{E}_{z\sim P^*}\left[\sup_{\mathbf{x}_{\text{adv}}\in N(\mathbf{x})} \mathbf{1}\big(h(\mathbf{x}_{\text{adv}}) \neq y\big)\right].$$

Attacks in Practice

- **Neighborhoods:** Typically defined as ℓ_p-norm balls

  $$N(\mathbf{x}) = \{\mathbf{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta} \mid \|\boldsymbol{\delta}\|_p \leqslant \varepsilon\},$$

  where ε is the perturbation budget.

- **Optimization Formulation:** In practice, the non-differentiable indicator loss $\mathbf{1}(h(\mathbf{x}_{\text{adv}}) \neq y)$ is replaced by a smooth proxy loss (e.g., cross-entropy). For a ***targeted attack*** toward target label T, the attacker solves:

  $$\min_{\|\boldsymbol{\delta}\|\leqslant\varepsilon} L_{\text{proxy}}\big((\mathbf{x}+\boldsymbol{\delta}, T), h\big).$$

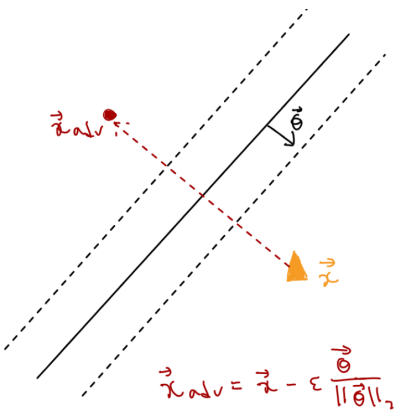
- **Optimization Method:** Typically solved using *Projected Gradient Descent (PGD)* or similar constrained optimization methods.

11.3 Attacking Linear Classifiers

For a two-class linear classifier with decision boundary defined by

$$\mathbf{w}^\top \mathbf{x} + b = 0,$$

the predicted label is determined by the sign of $\mathbf{w}^\top \mathbf{x} + b$.



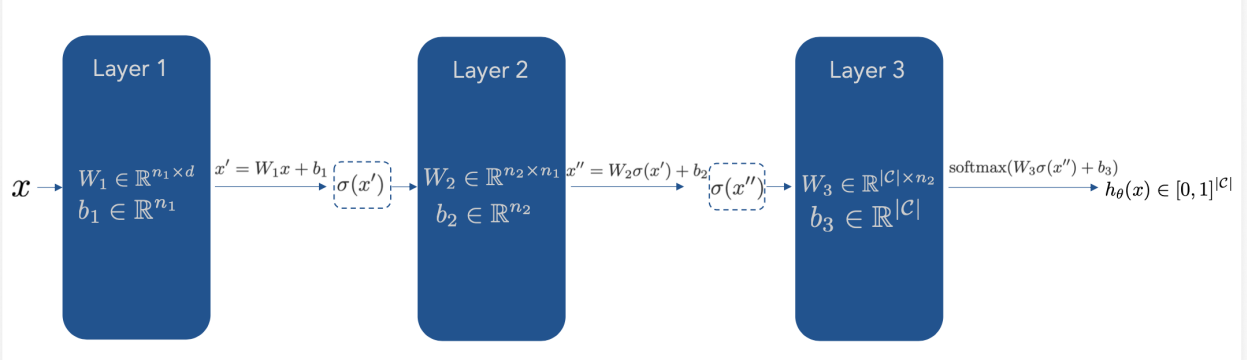
To flip the classification, we want to move the input $\mathbf{x}$ across the decision boundary with the smallest possible perturbation. Since the normal vector to the hyperplane is $\mathbf{w}$, the most effective direction to change the classifier's output is *orthogonal to the decision boundary*, i.e., along $-\mathbf{w}$. Hence, the adversarial example is chosen as

$$\mathbf{x}_{\text{adv}} = \mathbf{x} - \varepsilon \frac{\mathbf{w}}{\|\mathbf{w}\|_2},$$

where ε is the *amount of perturbation* or *adversarial budget*.

11.4 Overview: Neural Networks

A feedforward neural network applies a sequence of linear transformations and nonlinear activations to an input



where $\sigma(\cdot)$ is an element-wise non-linear function (e.g., ReLU)

$$\sigma(a) = \begin{cases} 0, & a < 0, \\ a, & a \geqslant 0, \end{cases} \quad \text{and} \quad \text{softmax}(a_i) = \frac{e^{a_i}}{\sum_j e^{\hat{a}_j}}.$$

and C is the number of classes.

The loss is denoted as

$$\ell((\mathbf{x}, y), h_\theta) = \ell\big((\mathbf{x}, y), h(W_1, b_1, W_2, b_2, W_3, b_3)\big).$$

Stochastic Gradient Descent

Each update step to the **parameters** to minimize the loss is given by:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla_{\boldsymbol{\theta}_t} \ell\big((\vec{x}, y), \boldsymbol{\theta}_t\big)$$

$$\text{where } (\vec{x}, y) \sim \mathscr{D}$$

Here, η is the learning rate that controls the step size, and each update is computed using a single (or mini-batch) sample drawn from the data distribution.

11.5 Single Gradient Ascent Step

Can we use the same idea as SGD to maximize loss over an input?
Yes, Instead of updating model parameters, we perturb the input in the direction that **increases** the loss the most. This leads to an **untargeted adversarial example**:

$$\vec{x}_{adv} = \vec{x} + \varepsilon \frac{\nabla_{\vec{x}} \ell((\vec{x}, y), h_{\boldsymbol{\theta}})}{\|\nabla_{\vec{x}} \ell((\vec{x}, y), h_{\boldsymbol{\theta}})\|_2}$$

where ε controls the size of the perturbation, often referred to as the *adversarial budget*.
Intuitively, this update takes a **single gradient ascent step** in input space — orthogonal to the classifier's decision boundary — to increase the loss.

Why?

When $\|\boldsymbol{\delta}\|_2 \leqslant \varepsilon$, we can approximate the loss at a perturbed point using a first-order Taylor expansion around $\vec{x}$:

$$\ell(\vec{x} + \boldsymbol{\delta}) = \ell(\vec{x}) + \boldsymbol{\delta}^\top \nabla_{\vec{x}} \ell(\vec{x}) + \mathcal{O}(\|\boldsymbol{\delta}\|^2)$$

Ignoring higher-order terms gives:

$$\ell(\vec{x} + \boldsymbol{\delta}) - \ell(\vec{x}) \approx \boldsymbol{\delta}^\top \nabla_{\vec{x}} \ell(\vec{x})$$

Expanding the dot product over dimensions:

$$\boldsymbol{\delta}^\top \nabla_{\vec{x}} \ell(\vec{x}) = \sum_{i=1}^{d} \delta_i \frac{\partial \ell(\vec{x})}{\partial x_i} \leqslant \sum_{i=1}^{d} |\delta_i| \left| \frac{\partial \ell(\vec{x})}{\partial x_i} \right|$$

By the Cauchy–Schwarz inequality:

$$|\boldsymbol{\delta}^\top \nabla_{\vec{x}} \ell(\vec{x})| \leqslant \|\boldsymbol{\delta}\|_2 \, \|\nabla_{\vec{x}} \ell(\vec{x})\|_2 = \varepsilon \, \|\nabla_{\vec{x}} \ell(\vec{x})\|_2$$

The maximum increase occurs when $\boldsymbol{\delta}$ is aligned with the gradient:

$$\boldsymbol{\delta} = \varepsilon \frac{\nabla_{\vec{x}} \ell(\vec{x})}{\|\nabla_{\vec{x}} \ell(\vec{x})\|_2}$$

Thus, the adversarial example is:

$$\vec{x}_{adv} = \vec{x} + \varepsilon \frac{\nabla_{\vec{x}} \ell((\vec{x}, y), h_{\boldsymbol{\theta}})}{\|\nabla_{\vec{x}} \ell((\vec{x}, y), h_{\boldsymbol{\theta}})\|_2}$$

Hence, move the input point in the direction of gradient because that *locally increases the loss the most*.

Question: What happens when we constrain $\|\delta\|_\infty \leqslant \varepsilon$ instead?

To answer this, recall the general inequality:

$$|\mathbf{a}^\top \mathbf{b}| \leqslant \|\mathbf{a}\|_p \|\mathbf{b}\|_q, \quad \text{where } \frac{1}{p} + \frac{1}{q} = 1.$$

This is a generalization of the Cauchy–Schwarz inequality (which corresponds to $p = q = 2$).
For the ℓ_∞ constraint, we have $p = \infty$ and $q = 1$, leading to:

$$\mathbf{a}^\top \mathbf{b} \leqslant \|\mathbf{a}\|_\infty \|\mathbf{b}\|_1.$$

Equality holds when $\mathbf{b}$ has the same sign pattern as $\mathbf{a}$ [1]. Thus, the perturbation that maximizes the loss is aligned with the sign of the gradient:

$$x_{\text{adv}} = x + \varepsilon \operatorname{sign}(\nabla_x \ell((\mathbf{x}, y), h_\theta)).$$

Each input feature is perturbed in the direction of the gradient's sign. Under an ℓ_∞ constraint, this causes the largest local increase in loss.

Targeted Adversarial Examples

Q: How do we move the example such that it is classified in some specific target class T?

In the untargeted setting, we sought perturbations that *maximize* the loss $\ell((\mathbf{x}, y), h_\theta)$, making the model more likely to misclassify the input. In contrast, a **targeted** adversarial attack aims to push the input *toward* a chosen target class T by *minimizing* the loss $\ell((\mathbf{x}, y_T), h_\theta)$, where y_T is the desired target label.

$$\mathbf{x}_{\text{adv}} = \mathbf{x} - \varepsilon \frac{\nabla_{\mathbf{x}} \ell((\mathbf{x}, y_T), h_\theta)}{\|\nabla_{\mathbf{x}} \ell((\mathbf{x}, y_T), h_\theta)\|_p}$$

This formulation mirrors the untargeted case, except that the gradient direction is reversed, we now move *opposite* to the direction that increases loss for the target label.

Intuition: The targeted perturbation shifts the input so that the model becomes increasingly confident in the chosen target class T. In geometric terms, the example is moved across the decision boundary and pulled toward the region corresponding to the target class.

11.6 White-box Adversarial Examples and Projected Gradient Descent

In a **white-box attack**, the adversary has complete access to the model, including its parameters θ, architecture, and gradients. This allows direct computation of the loss gradient with respect to the input, $\nabla_x \mathscr{L}(h_\theta(x), y)$, enabling precise perturbations that maximize (or minimize) the loss.

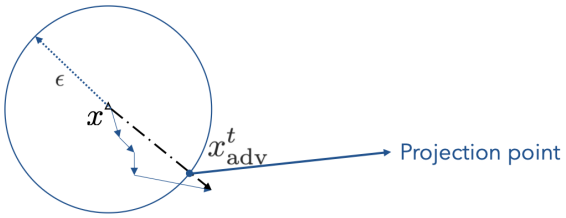


Figure 11.2. White-box adversarial example generation: the adversarial point is iteratively updated and projected back to the ε-ball.

The iterative update rule for the adversarial example can be written as:

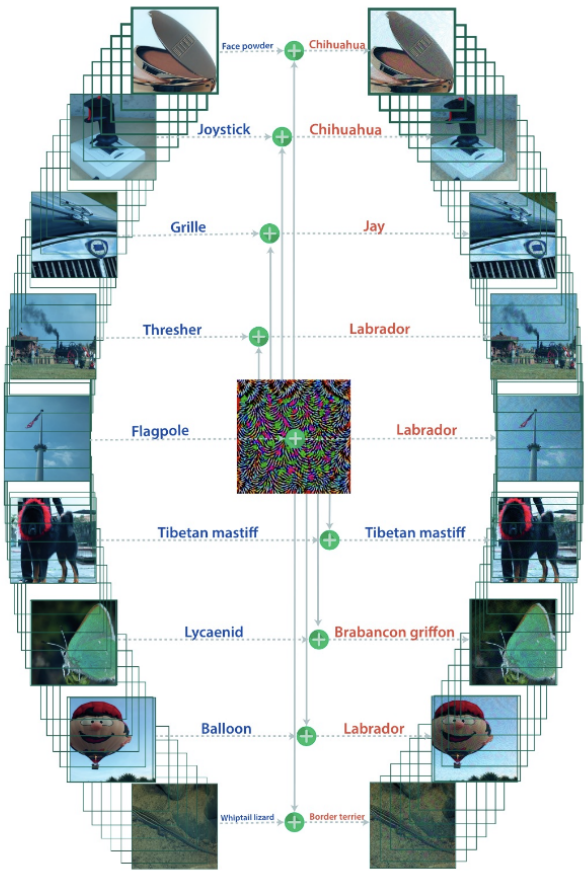
$$\mathbf{x}_{\text{adv}}^{t} = P_{\mathbf{x},\varepsilon}\left(\mathbf{x}_{\text{adv}}^{t-1} + \alpha \frac{\nabla_{\mathbf{x}_{\text{adv}}^{t-1}} \mathrm{l}((\mathbf{x}_{\text{adv}}^{t-1}, y), h_{\boldsymbol{\theta}})}{\|\nabla_{\mathbf{x}_{\text{adv}}^{t-1}} \mathrm{l}((\mathbf{x}_{\text{adv}}^{t-1}, y), h_{\boldsymbol{\theta}}\|_2}\right)$$

where $P_{\mathbf{x},\varepsilon}$ projects the adversarial example back onto an ε-ball around the original input $\mathbf{x}$.

Intuition: Search iteratively for the empirically strongest perturbation by taking many small gradient steps and projecting back into the allowed ε-ball.

11.7 Universal Adversarial Perturbations[3]

Key Question: What if we want a single adversarial perturbation that works for multiple different inputs?



The Universal Adversarial Perturbation (UAP) algorithm seeks a single, image-agnostic perturbation that can fool a classifier across many inputs. It iteratively updates a universal perturbation vector $\boldsymbol{\delta}$ by

accumulating minimal input-specific perturbations $\mathbf{v}_i$ that cause misclassification. After each update, $\boldsymbol{\delta}$ is projected back onto the allowed ℓ_p-ball to keep the perturbation imperceptible, resulting in a universal perturbation that generalizes across samples.

Initialize $\boldsymbol{\delta}^{(0)} = \mathbf{0}$
for each input $\mathbf{x}_i$ **do**
  if $h_\theta(\mathbf{x}_i + \boldsymbol{\delta}^{(n)}) = h_\theta(\mathbf{x}_i)$ **then**
    $\mathbf{v}_i \leftarrow \arg\min_{\|\mathbf{r}\|_2 \leqslant \xi} h_\theta(\mathbf{x}_i + \boldsymbol{\delta}^{(n)} + \mathbf{r}) \neq h_\theta(\mathbf{x}_i)$
    $\boldsymbol{\delta}^{(n+1)} \leftarrow P_{\mathbf{x},\varepsilon}\left(\boldsymbol{\delta}^{(n)} + \mathbf{v}_i\right)$
  end if
end for

11.8 Types of Adversarial Attacks

Below are common categories of adversarial attacks (brief descriptions):

1. **Neighborhood / norm-bounded attacks**
   Perturbations constrained to a small neighborhood around the input, e.g.

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta}, \qquad \|\boldsymbol{\delta}\|_p \leqslant \varepsilon.$$

2. **Minimum-norm (optimization) attacks**
   Find the smallest perturbation that causes misclassification:

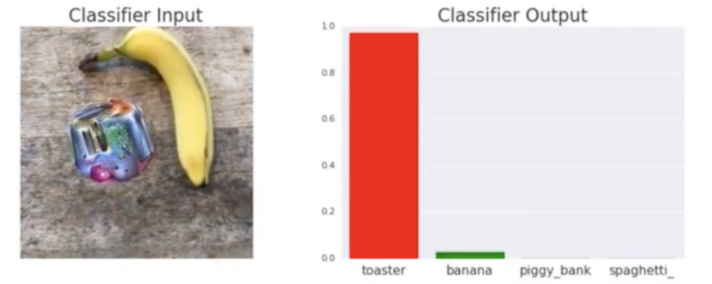
$$\min_{\boldsymbol{\delta}} \|\boldsymbol{\delta}\|_2 \quad \text{s.t.} \quad h_\theta(\mathbf{x} + \boldsymbol{\delta}) \neq y.$$

   These attacks (e.g. Carlini–Wagner) seek perceptually minimal changes that flip the label.

3. **Patch-based attacks**
   Localized (often visible) perturbations restricted to a region or patch M:

$$\mathbf{x}_{\text{adv}} = \mathbf{x} + \boldsymbol{\delta}, \qquad \text{supp}(\boldsymbol{\delta}) \subseteq M(\mathbf{x}),$$

   where $\boldsymbol{\delta}$ is nonzero only on a patch (e.g. stickers, adversarial patches) and $M(\mathbf{x})$ denotes the allowed patch mask/placement.

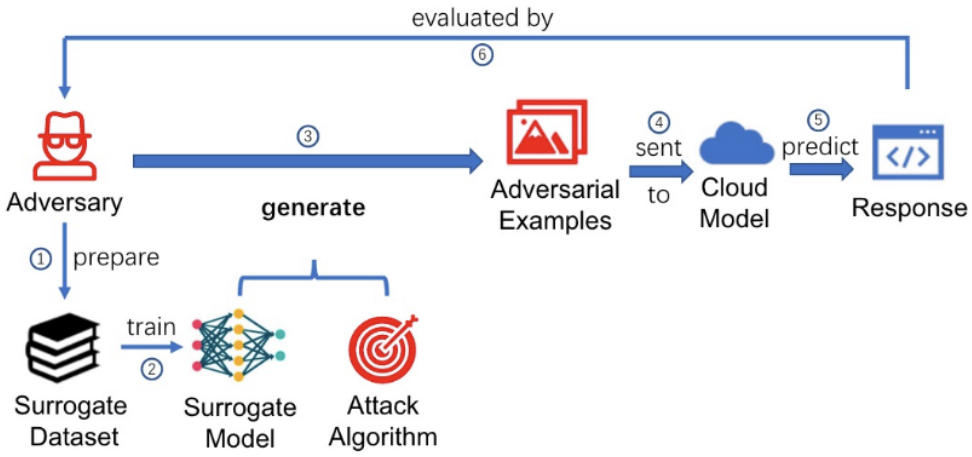


Adversarial Patches

4. **Black-box / query-based attacks**
   The adversary does not have access to model internals; attacks rely on queries:

   - *Score-based* (or confidence-based): attacker queries the model and uses returned scores/probabilities to estimate gradients or perform optimization.
   - *Decision-based*: attacker only observes final labels (top-1); algorithms perform boundary-search or estimate minimal perturbations via queries.

   Black-box methods include transfer-based attacks (craft on surrogate models) and iterative query strategies [2] .



Bibliography

[1] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2015. Original FGSM paper deriving the ℓ_∞ step as the Fast Gradient Sign Method.

[2] Chuan Guo Mao, Yisen He, Shiqi Xu, Cihang Xie, and Alan L. Yuille. Adversarial attacks are reversible with natural supervision. In *Proceedings of the IEEE Symposium on Security and Privacy (S&P)*, pages 1773–1789. IEEE, 2022.

[3] Seyed-Mohsen Moosavi-Dezfooli, Alhussein Fawzi, and Pascal Frossard. Universal adversarial perturbations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1765–1773, 2019.